# Authorship Attribution at the Frontier: Dataset Architecture, Agentic Training in Google Antigravity, and Transductive Ensembling for the Name That LLM Competition

## Executive Summary and Problem Formulation

The Kaggle competition **Name That LLM** poses a fine-grained machine-generated text (MGT) authorship attribution challenge across 18 frontier large language models originating from six distinct AI laboratories: OpenAI, Google, Anthropic, Qwen, ZAI, and Moonshot AI[1]. Unlike conventional binary AI detection, which merely separates synthetic prose from human writing, this task requires pinpointing the exact model version from a single, self-contained paragraph ranging between 40 and 250 words across unseen prompts in philosophy, physics, mathematics, biology, computer science, and creative prose[1].

The competition evaluates submissions using **Balanced Accuracy**, defined as the unweighted macro-average of per-class recall across all $K = 18$ candidate models[1]:

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_{k=1}^{K} \frac{|\{i : y_i = k \wedge \hat{y}_i = k\}|}{|\{i : y_i = k\}|}$$

While random guessing across 18 classes yields a baseline score of $1/18 \approx 0.0556$, early naive baselines hovered around $0.330$ to $0.353$, and leading multi-stage ensembles on the public leaderboard have escalated to between $0.659$ and $0.730$ [2]. Achieving a top-tier score requires resolving three fundamental structural asymmetries embedded in the competition design: sample scarcity combined with intra-lab sibling confusion, topic-confounded cross-validation, and class prior shifts between the training and hidden test splits[1].

The official training set (train.csv) contains exactly 2,699 passages distributed across 150 unique prompts (prompt_id), where 149 prompts are answered once by all 18 models and one prompt (p002) is answered by 17 models, omitting claude-fable-5[6]. With only $\sim 150$ examples per class, standard neural fine-tuning struggles to separate generational siblings within the same laboratory—most notably the four Google Gemini models (gemini-3.1-pro-preview, gemini-3.6-flash, gemini-3.7-flash, gemini-3.8-flash), the five Anthropic Claude models (claude-fable-5, claude-opus-4-6, claude-opus-4-8, claude-opus-5, claude-sonnet-5), and the five OpenAI models (gpt-5.5, gpt-5.6-luna, gpt-5.6-sol,

gpt-5.6-terra, gpt-6-astra)[1]. Out-of-fold error analysis demonstrates that while provider-level attribution reaches $0.587$ to $0.685$ accuracy using surface stylometry alone, roughly half of all misclassifications occur between sibling models from the same provider, and ZAI's glm-5.3 and glm-5.3-flash frequently masquerade as claude-opus-5[5].

Because every training prompt is answered by all 18 models, evaluating models via standard random StratifiedKFold introduces severe negative topic bias[6]. When a passage is held out for validation in a random split, its 17 sibling passages—sharing the exact same rare topic vocabulary—remain in the training fold attached to the *other* 17 labels[6]. Consequently, a classifier memorizes topic words as negative evidence against the true authoring model, causing random cross-validation to severely underestimate performance ($0.217$ random CV versus $0.369$ prompt-grouped CV on the exact same model, where the public leaderboard scores $0.355$)[6]. Furthermore, while the training set is almost perfectly balanced across prompts and models, the competition hosts explicitly state that the hidden test set ($\sim 4,000$ passages) features unequal passage counts per model and that the class mix differs between the public and private leaderboards[1]. Therefore, naive uniform marginal forcing or hard one-per-prompt assignment degrades leaderboard performance, necessitating covariance-shrunk transductive centering and target-shrunk Sinkhorn-Knopp calibration[6].

| **Provider** | **Candidate Model Label** | **Architectural Tier & Post-Training Profile** | **Primary Stylometric & Behavioral Signatures** |
|---|---|---|---|
| **OpenAI** | gpt-5.5 | Legacy 5th-gen generalist model[1] | Curly apostrophes (\u2019); structured explanatory cadence; high data-analysis precision[10]. |
| **OpenAI** | gpt-5.6-luna | High-volume, cost-sensitive nano-tier model ($0.20/MTok input, 1.05M context)[12] | Curly apostrophes (\u2019); compact phrasing; strong coding/math point estimates (58.5% FrontierMath Tier 4)[11]. |
| **OpenAI** | gpt-5.6-sol | Software | Curly apostrophes |

| | | | |
|---|---|---|---|
| | | engineering & deep problem-solving specialist[10] | (\u2019); dense logical transitions; high LiveBench deduction (91.7) and coding (83.9)[10]. |
| **OpenAI** | gpt-5.6-terra | Enterprise workflow & document automation model ($2.00/MTok input)[12] | Curly apostrophes (\u2019); direct, declarative prose optimized for routine office and document tasks[11]. |
| **OpenAI** | gpt-6-astra | Next-gen frontier flagship[1] | Curly apostrophes (\u2019); top-tier analytical deduction (92.7 LiveBench) and mathematics (96.8); distinct clause subordination[10]. |
| **Google** | gemini-3.1-pro-preview | Heavyweight multimodal preview[1] | Curly apostrophes (\u2019); high instruction-following adherence (79.1 LiveBench); expansive vocabulary[10]. |
| **Google** | gemini-3.6-flash | Mid-cycle high-efficiency Flash iteration[1] | Curly apostrophes (\u2019); cautious, structured phrasing; high lexical overlap with 3.7/3.8 siblings[7]. |
| **Google** | gemini-3.7-flash | Late-August 2026 accelerated Flash model[1] | Curly apostrophes (\u2019); 79.9 LiveBench instruction following; subject |

| | | | |
|---|---|---|---|
| | | | to API auto-routing contamination in unverified logs[10]. |
| **Google** | gemini-3.8-flash | September 2026 agentic & cyber-hardened Flash workhorse ($0.75/MTok input)[19] | Curly apostrophes (\u2019); leads instruction following (81.4 LiveBench) and ARC-AGI-2 (89.2%); scannable, answer-first organization[10]. |
| **Anthropic** | claude-fable-5 | 5th-gen "Mythos-level" asynchronous research flagship ( $10/50 MTok)[22] | Straight apostrophes ('); adaptive thinking always-on (high); leads language benchmarks (90.7 LiveBench); occasional safety refusals[10]. |
| **Anthropic** | claude-opus-4-6 | Early 4th-gen Opus frontier model[1] | Straight apostrophes ('); classic literary Anthropic cadence; moderately separable via within-Claude linear gates (0.59 recall with extra data)[11]. |
| **Anthropic** | claude-opus-4-8 | Late 4th-gen Opus refinement[1] | Straight apostrophes ('); shares 9.3x multilingual token overhead profile with Fable 5; high stylistic proximity to Opus 4.6[11]. |

| | | | |
|---|---|---|---|
| **Anthropic** | claude-opus-5 | 5th-gen deep knowledge & literary model[1] | Straight apostrophes ('); highly distinctive syntax achieving $0.66–0.73$ recall; frequently mimics or attracts false positives from ZAI GLM models[7]. |
| **Anthropic** | claude-sonnet-5 | High-efficiency 5th-gen balanced enterprise model[1] | Straight apostrophes ('); crisp, balanced rhythm; recall jumps from $0.43$ to $0.66$ when augmented with API-matched Claude passages[11]. |
| **Qwen** | qwen3.8-max | Alibaba flagship MoE/dense frontier model[1] | Curly apostrophes (\u2019); 78.5 LiveBench overall; distinct punctuation spacing and transitional adverbials[10]. |
| **ZAI** | glm-5.3 | Flagship Z.AI frontier model[1] | Straight apostrophes ('); strong stylistic convergence toward Anthropic claude-opus-5, requiring deep hidden-state representations to disambiguate[7]. |
| **ZAI** | glm-5.3-flash | High-speed distilled/compact | Straight apostrophes ('); |

| | | | |
|---|---|---|---|
| | | Z.AI model[1] | shorter clause structures than glm-5.3 but shares ZAI typographic and lexical priors[7]. |
| **Moonshot AI** | kimi-k3 | Moonshot AI long-context flagship (79.2 LiveBench)[1] | Distinct lexical and rhythm fingerprint; sensitive to provider routing differences in third-party datasets[10]. |

# Comprehensive Audit of the External Dataset Ecosystem

Under Section 2.6 of the competition rules, freely and publicly available external datasets—including synthetic passages generated via vendor APIs and released under permissive open licenses—are permitted for model training[1]. Because the official training set provides only 2,699 passages, the competitive landscape has been defined by large-scale collaborative data generation[9]. However, empirical ablations across the leaderboard have proven a critical second-order finding: **data purity and generation protocol alignment vastly outweigh raw corpus volume**[9].

When competitors naively concatenated scraped evaluation logs or uncleaned public pools into their training pipelines, cross-validation and leaderboard scores actively degraded[9]. For example, retraining the Qwen2.5-14B backbone of the nlm-v27 ensemble on an uncurated 120k passage pool increased out-of-fold validation from $0.569$ to $0.605$ but caused the public leaderboard score to drop from $0.651$ to $0.647$ [9]. Conversely, when Carrot Production audited the public pool (Name That LLM Open Pool v9 and v10), the audit revealed that **1 in 4 passages (roughly 25%) in the raw 255k pool was junk or belonged to the wrong genre**—containing markdown headers, bulleted lists, multi-paragraph essays, code blocks, or API refusal boilerplate rather than a single self-contained paragraph of 40 to 250 words[9]. Filtering the 255k pool down to **198k clean, protocol-matched passages** yielded an immediate **+0.027 balanced accuracy improvement** using the exact same model architecture[9].

KAGGLE CORPUS INTELLIGENCE October 2026 Audit

# Curated Protocol-Matched Pools Outperform Raw Scraped Corpora by +0.027 Balanced Accuracy

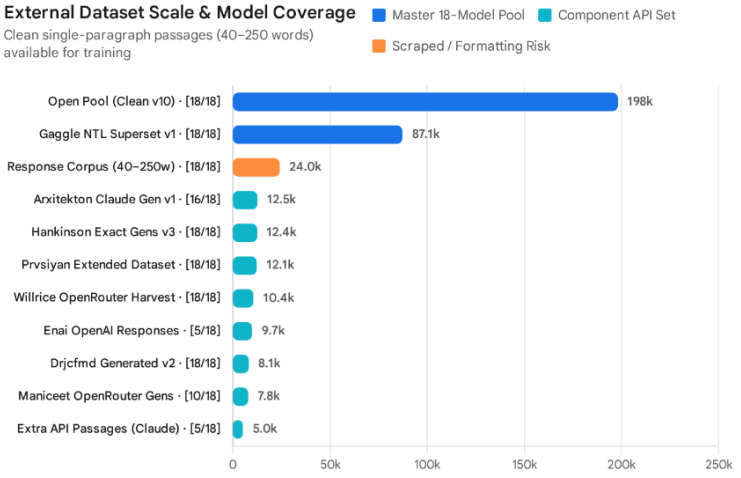
The official competition training set contains just 2,699 passages across all 18 models. High-scoring pipelines ingest multi-source synthetic corpora, but filtering out routing errors and formatting artifacts is essential for out-of-fold generalization.

| OFFICIAL TRAIN | OPEN POOL V10 CLEAN ● |
|---|---|
| 2,699 | 198,000 |
| 18 / 18 Models · Baseline | 18 / 18 Models · +0.027 Acc |

| OPEN POOL RAW | GAGGLE SUPERSET |
|---|---|
| 255,000 | 87,106 |
| 18 / 18 Models · 57k Flagged | 18 / 18 Models · CC BY-SA 4.0 |

## External Dataset Scale & Model Coverage

Clean single-paragraph passages (40–250 words) available for training




* Passages drawn exclusively from self-contained prompts matching official prompt instructions Labels rendered with exact sample count

### Open Pool Contamination Audit

57,000 Purged

Raw community aggregations contain model proxy misroutes and prompt leakage that degrade out-of-fold cross-validation.

| | |
|---|---|
| **flag_gemini37_routing_risk** Provider auto-routed 3.7 to 3.8 flash | 8,520 rows |
| **flag_near_train_prompt** TF-IDF cosine similarity to train prompt > 0.3 | 323 rows |
| **Formatting / Length Anomalies** Markdown lists, titles, <40w or >250w passages | ~48,157 rows |

Filter: `df[~df[flags].any(axis=1)]` 198k Zero-Flag Rows

### Ablation Lift (539 Held-Out Rows)

+0.095 Max Δ

Empirical impact on Qwen3-8B LoRA & LinearSVC prompt-grouped cross-validation when integrating balanced synthetic data.

| | |
|---|---|
| Public LoRA24 Plain (Train Only) | 0.456 |
| + Claude Head | 0.484 (+0.028) |
| + In-Style Extra Protocol Passages | 0.516 (+0.032) |
| Blended Public + In-Style LoRA Ensemble | 0.551 (+0.035) |

Classifier Baseline (LinearSVC): 0.372 → 0.399 (+0.027)

Comparison of publicly licensed external datasets for the Name That LLM competition as of late October 2026. Filtering out formatting mismatches, prompt leakage, and API routing risks in the 255k Open Pool yields 198k clean passages across all 18 models and boosts balanced accuracy by +0.027.

Data sources: Carrot Production (Open Pool v10), Gaggle NTL Superset v1, In-Style Qwen3-8B LoRA Benchmark, Seyit Kaan Güneş (Claude Passages), Maniceet OpenRouter Harvest, Oleg Zholobov 18-Model Corpus

## Tier 1: Master Unified Datasets Covering All 18 Models

To train a competitive model in Google Antigravity without manually downloading and reconciling thirty separate repositories, practitioners should anchor their data pipeline on three audited, multi-model master datasets: **Name That LLM Open Pool**, **prvsiyan's Extended Name That LLM Dataset**, and **Gaggle NTL Superset v1**[11].

The primary large-scale resource is **Name That LLM Open Pool** (crtproduction / Carrot Production), distributed as /kaggle/input/**/ntl_open_pool.parquet[20]. It encompasses **255,000 total passages** across all 18 competition models in its raw release and **198,000 clean passages** (86 MB, Usability 9.4) in its v10 filtered release, ensuring that every single one of the 18 candidate models is represented by thousands of passages[9]. The dataset merges openly licensed public datasets shared by competitors, strips surrounding whitespace, filters word counts strictly to match the competition's 40–250 word distribution, and removes any exact or near copies of the official training data by enforcing a word TF-IDF cosine similarity threshold of max_cos_train < 0.3 against train.csv[20]. Crucially, each row includes provenance metadata (source_dataset, also_in, n_sources, and prompt_group for grouped cross-validation) alongside three boolean contamination flags: flag_gemini37_routing_risk (8,520 rows), which identifies passages labeled gemini-3.7-flash originating from third-party proxy sources where Google's API may have silently routed requests to gemini-3.8-flash after its September 2, 2026 release[19]; flag_near_train_prompt (323 rows), which flags passages whose underlying prompt topic is dangerously close to an official train.csv prompt to prevent topic-to-label leakage[20]; and flag_train_prompt_regen, which marks synthetic regenerations that directly reused a competition training prompt[20]. Executing the one-line filter clean(pool) = pool[~pool[flags].any(axis=1)] immediately yields the 198k high-purity corpus under a CC BY-SA 4.0 license[20].

Complementing the massive Open Pool is **prvsiyan's Extended Name That LLM Dataset** (prvsiyan/prvsiyans-extended-name-that-llm-dataset) and its companion NTL public in-style pool v1, which provide **9,290 to 12,142 API-logged passages** across 510+ prompts covering **all 18 candidate models** (passages.csv / nlm_matched_clean.parquet, 22 MB, Usability 9.4, CC BY-SA 4.0 / CC BY 4.0)[2]. Unlike massive heterogeneous pools, prvsiyan's dataset was generated under a tightly controlled, balanced multi-model prompt design where nearly every prompt is answered by all 18 models at matched target word counts[11]. It serves as the primary external training set for the ntl-qwen3-8b-lora-instyle checkpoint—which lifted single-model held-out balanced accuracy from $0.456$ to $0.516$ —as well as the stylometric-4view-sota pipeline[28]. Empirical testing indicates that when training neural LoRA heads on early blocks of this dataset, dropping its kimi-k3 rows and supplementing kimi-k3 from maniceet or arxitekton further improves calibration[27].

A third master option is **Gaggle NTL Superset v1** (riccapucho/gaggle-ntl-superset-v1), stored as gaggle_ntl_superset_v1.parquet and containing **87,106 labeled passages** across all 18 models[34]. Synthesized from 28 original public datasets and three curated intermediate pools (nishantdahal/ntl-handwriting-blend, pukkei37/ntl-pool-v2, and ahmedberatozer/ntl-pool-v4), this dataset provides a mid-sized, license-audited alternative to the 255k Open Pool[34]. The

authors traced every row to its original dataset, stripped 1,455 rows with unverified licenses, and completely excluded olegzholobov's scraped corpus due to formatting and relicensing constraints[34].

## Tier 2: High-Purity Provider and Host-Protocol Component Datasets

For specialized second-stage training—such as fitting hierarchical within-lab classifiers ("Claude gates" or "Gemini Flash gates") or constructing balanced multi-model batches where every prompt has $\geq 12$ model responses—several high-purity component datasets provide superior signal-to-noise ratios[27].

| Dataset Slug (kaggle.com/datasets/...) | Passage Count | Model Coverage | Generation Protocol & Strategic Utility | License |
|---|---|---|---|---|
| crtproduction/name-that-llm-open-pool | 198,000 clean (255,000 raw)[32] | All 18 models[20] | Master deduplicated pool merged from all open public sources; filtered to host word-count range with explicit routing and prompt-leakage flags (ntl_open_pool.parquet)[20]. | CC BY-SA 4.0[20] |
| riccapucho/gaggle-ntl-superset-v1 | 87,106[34] | All 18 models[34] | License-audited consolidation of 28 public datasets via 3 intermediate pools; excludes unverified and scraped evaluation logs (gaggle_ntl_superset_v1.parq | CC BY-SA 4.0[34] |

| | | | | |
|---|---|---|---|---|
| | | | uet)[34]. | |
| arxitekton/ntl-claude-gen-v1 & v2 | 15,777 (12,476 v1 + 3,301 v2)[20] | 16 of 18 models[39] | Generated across 1,060 new prompts using pinned first-party API routes on OpenRouter following the exact host length protocol[39]. Ideal for multi-way shared-prompt Gaussian fitting. | CC BY 4.0[34] |
| elliotthankinson/ntl-exact-gens-v3 | 12,378[20] | All 18 models[20] | Strictly verifies that the served_model string returned in the API response header matches the requested competition model ID[20]. Eliminates proxy routing noise. | Open / Public[20] |
| prvsiyan/prvsiyans-extended-name-that-llm-dataset | 12,142 (9,290 v1)[11] | All 18 models[11] | Balanced API-logged corpus across 510+ prompts where all 18 models answer each prompt (passages.csv / | CC BY-SA / CC BY 4.0[11] |

| | | | | |
|---|---|---|---|---|
| | | | nlm_matched_clean.parquet)[2]. | |
| willrice/name-that-llm-openrouter-harvest | 10,403[34] | All 18 models[34] | High-usability (10.0) clean OpenRouter harvest across candidate models with full JSON/CSV provenance[34]. | CC BY 4.0[34] |
| maniceet/name-that-llm-openrouter-openai-generations | 7,753[27] | 10 non-Claude models (gpt-*, glm-*, kimi-k3, qwen3.8-max, gemini-3.7-flash)[27] | Uses explicit prompt "Write a single paragraph of about N words, with no title or headings" with $N$ sampled from train.csv word counts[27]. Crucially pins the official **Z.AI** provider for glm-5.3 and glm-5.3-flash with zero fallbacks, solving ZAI attribution[27]. | CC BY 4.0[27] |
| seyitkaangunes/ntl-extra-api-passages | 4,998[24] | 5 Anthropic Claude models (claude-*)[24] | 1,000 new agent-written prompts $\times$ 5 Claude models via official Anthropic Message Batches (2 | CC BY 4.0[24] |

| | | | | |
|---|---|---|---|---|
| | | | refusals on claude-fable-5 dropped)[24]. Essential for training a specialized 5-way **Claude Gate**[38]. | |
| drjcfmd/ntl-generated-v2 & lxdlxdlxd/ntl-synth-generations-v1 | 12,517 (8,111 + 4,406)[34] | 18 models (multi-family)[34] | Clean synthetic single-paragraph generations widely incorporated into top two-stage LoRA pools[27]. | CC BY-SA / CC BY 4.0[34] |
| alperen5252525/ntl-vertex-gemini-proxy-gpt-generations & lucasboesen/ntl-free-proxy-generations | 6,015 (3,346 + 2,669)[34] | OpenAI gpt-* & Google gemini-*[28] | Generated via Google Vertex AI and the Kaggle Benchmarks Model Proxy; provides strong coverage for separating the 5 GPT and 4 Gemini siblings[28]. | CC BY 4.0[34] |

## Tier 3: Auxiliary Benchmarks and Quarantined Corpora

Two additional external resources appear frequently in competition discussions but require specialized handling to prevent model degradation[18]. The first is **olegzholobov/name-that-llm-response-corpus**, which contains 135,171 unique responses across 70,980 wide rows, including 24,015 candidates in the 40–250 word window and 1,902 prompt groups answered by all 18 models[31]. Because this corpus was compiled from heterogeneous public Hugging Face evaluation logs, GitHub repositories, Artificial Analysis, and Novigens, empirical testing by multiple competitors confirmed that adding even the length-filtered subset (external_short_wide.parquet) directly to training **hurts**

**cross-validation**[31]. This degradation stems from a severe formatting gap—embedded newlines, Markdown syntax, categorical answers, and varying system prompts—alongside weaker label verification on gpt-5.6-luna and non-relicensed source rows[31]. Consequently, it should be used strictly for exploratory vocabulary analysis and never mixed raw into supervised classifiers[31].

The second resource is **MAABench and ATLAS Pretraining (ming9999/maabench on Hugging Face)**, introduced alongside the October 2026 paper *"Who's Behind the Text? Robust Model Authorship Attribution"* (arXiv:2610.19583)[18]. MAABench contains prompt-paired generations targeting 90–220 words across six domains (News, Wikipedia, Academic Writing, Product Review, Patent, and Stack Exchange) for five model families[44]. However, its version coverage only partially overlaps the 18 competition labels (gpt-5.6-terra, claude-opus-4-6, claude-opus-4-8, claude-opus-5, and gemini-3.6-flash, alongside older non-competition versions like gpt-5 and gemini-2.5-flash)[18]. Rather than mixing MAABench rows into the 18-way classifier, competitors leverage the **ATLAS** architecture (Joseph Adamski's NTL ATLAS Qwen3 8B v1 dataset on Kaggle) as an auxiliary 10% ensemble component for uncertain test rows[18].

# Agentic Development and Training Workflow in Google Antigravity

To process nearly 200,000 external passages, engineer multi-view stylometric features, and fine-tune 8B to 14B parameter LoRA backbones efficiently before uploading inference weights to Kaggle, **Google Antigravity** serves as the ideal orchestration environment[47]. Following the **Google Antigravity 2.0** platform expansion in May 2026, the ecosystem comprises the standalone desktop application, the VS Code-based **Antigravity IDE**, the Go-based **Antigravity CLI** (v2.28.2, which permanently replaced the retired Gemini CLI in June 2026), and the Python **Antigravity SDK** (v0.1.23) for programmatic multi-agent orchestration on local or cloud GPU infrastructure[17].

## Critical Environment Safeguard: Avoiding the Antigravity IDE v2.6.0 Regression

Before initiating model development in late October 2026, practitioners must address a critical software regression in **Antigravity IDE v2.6.0**[52]. In version 2.6.0, the interactive diff view suffers from a severe defect wherein autonomous agents write changes directly to disk without displaying modified files in the "files changed" panel, removing the explicit Accept/Reject confirmation buttons and automatically committing or discarding agent edits whenever a file is saved or closed[52]. In a machine learning competition where an agent silently altering a cross-validation split or label mapping can invalidate a 9-hour GPU run, this behavior poses a major operational risk[9].

To maintain strict codebase integrity, practitioners should either downgrade and pin **Antigravity IDE v2.5.5** directly from https://antigravity.google/releases with automatic updates disabled in the settings—as version 2.5.5 preserves full interactive diff review and cautious multi-file coordination—or operate via the official **VS Code Extension (v1.10.0)** paired with the **Antigravity CLI (v2.28.2)** (curl -fsSL https://antigravity.google/cli/install.sh | bash)[51]. When

configuring the workspace, selecting the **"Review-driven development"** autonomy profile ensures that agents pause at checkpoints and require explicit human approval before executing terminal commands[48].

## Decoupled Multi-Agent Training and Offline Kaggle Inference Architecture

Antigravity's core advantage in machine learning experimentation is its **Agent Manager (Mission Control)** and **Antigravity SDK**, powered by **Gemini 3.8 Flash** (which achieves 89.4% on Terminal-Bench 2.1 and outperforms larger frontier models on long-horizon software engineering in DeepSWE v1.1) alongside **Claude Sonnet 5.5 / Opus 5.5** and newly integrated OpenAI models[13]. Because Google removes claude-opus-4-6 and claude-sonnet-4-6 from the Antigravity model selector on November 2, 2026, agent profiles should be pinned to **Gemini 3.8 Flash** for rapid repository orchestration, data pipelines, and terminal execution, paired with **Claude Opus 5.5** or **Claude Fable 5.1** for complex mathematical verification of Gaussian covariance shrinkage[14].

Because Kaggle submissions must execute offline within 9 hours on a 2x NVIDIA T4 GPU kernel, the optimal system architecture decouples heavy multi-agent training in the **Google Antigravity 2.0 Workspace** from lightweight, transductive inference in the **Kaggle Offline 2x T4 Kernel**[1]. By defining an AGENTS.md configuration file in the root of the Antigravity workspace, a single developer can coordinate three isolated, asynchronous background agents in **Agent Manager** that write serialized model checkpoints into a unified **Kaggle Dataset Artifacts** store[17].

**Workspace Agent 1 (Open Pool Filter & Curation Agent)** utilizes the Antigravity CLI and Kaggle API to ingest competitions/name-that-llm, crtproduction/name-that-llm-open-pool, prvsiyan/prvsiyans-extended-name-that-llm-dataset, maniceet/name-that-llm-openrouter-openai-generations, and seyitkaangunes/ntl-extra-api-passages[20]. This agent executes automated deduplication, drops all rows flagged by flag_gemini37_routing_risk, flag_near_train_prompt, and flag_train_prompt_regen, enforces single-paragraph formatting (40–250 words, zero markdown or newlines), and exports a unified StratifiedGroupKFold index grouped strictly by prompt_id for official train rows and prompt_group for external pool rows[6].

Concurrently, **Workspace Agent 2 (Multi-Layer LoRA Backbone Trainer)** orchestrates multi-GPU training scripts (torchrun) for **Qwen3-8B-Base**, **Qwen3-14B-Base**, **Qwen2.5-14B**, and **DeBERTa-v3-large**[2]. As documented in competition debugging logs, skipping a non-finite gradient batch naively in torchrun DistributedDataParallel (DDP) causes rank desynchronization and hangs a 9-hour run; the agent therefore enforces synchronized gradient clipping and finite-loss masking across all ranks before exporting merged LoRA weights and per-layer LayerNorm heads (heads.safetensors)[9].

In parallel, **Workspace Agent 3 (Stylometry & Specialist Gate Fitter)** extracts the 81 deterministic typographic features (ntl_style.py) plus character/word TF-IDF matrices, fits the LinearSVC and TabPFN heads, fits the 5-way Anthropic claude_head and 4-way gemini_head, verifies numerical parity on anchor rows (anchor_features.npz), and packages the linear

readout matrices (readouts.npz, COMPLETE.json) into the Kaggle Dataset[11].
During the **Kaggle Offline 2x T4 Inference** stage, the submission notebook first executes a **Sequential Disk Copy to /tmp** to bypass /kaggle/input mmap bottlenecks[2]. It then evaluates the hidden test set (~4,000 passages) across three parallel inference branches—**Branch A** (81-Feature Stylometry + Character/Word TF-IDF SVC), **Branch B** (Two-Stage Qwen 8B/14B & DeBERTa-v3-large LoRA with Specialist Claude Gates), and **Branch C** (ATLAS Token Likelihoods on uncertain rows)—before converging into the **Shared-Prompt Gaussian & Adaptive Centering** module and **Target-Shrunk Sinkhorn-Knopp Calibration** to output submission.parquet[1].

| Execution Zone | Component / Agent Module | Assigned Model / Tooling | Primary Input Data / Artifacts | Exported Output & Downstream Role |
|---|---|---|---|---|
| **Zone 1: Google Antigravity 2.0 Workspace (Online Training)** | **Agent 1: Open Pool Filter & Curation** | Antigravity CLI v2.28.2 + Gemini 3.8 Flash[14] | train.csv (2.7k), ntl_open_pool.parquet (255k), prvsiyan (12.1k), maniceet (7.7k), seyitkaangunes (5k)[20] | 198k clean protocol-matched corpus + 5-fold StratifiedGroupKFold split indices by prompt_id / prompt_group[6]. |
| **Zone 1: Google Antigravity 2.0 Workspace (Online Training)** | **Agent 2: LoRA Backbone Trainer** | Antigravity SDK v0.1.23 + torchrun DDP[9] | train.csv + Clean multi-model prompts ( $\geq 12$ models/prompt)[28] | Multi-layer pooled LoRA adapters (Qwen3-8B/14B, Qwen2.5-14B, DeBERTa-v3-large) + heads.safetensors[2]. |
| **Zone 1: Google Antigravity 2.0** | **Agent 3: Stylometry & Gate Fitter** | Claude Opus 5.5 / Fable 5.1 + Scikit-Learn[17] | Multi-layer pooled embeddings $Z$ , ntl_style.py | readouts.npz (plain_*, center_*, claude_*), |

| | | | | |
|---|---|---|---|---|
| **Workspace (Online Training)** | | | (81 features), 5k Claude API passages[11] | gauss__*.npz, and anchor_features.npz uploaded as Kaggle Dataset[28]. |
| **Zone 2: Kaggle Offline 2x T4 Kernel (Scoring Rerun)** | **Stage 1: Sequential Disk Staging** | Python shutil.copytree to /tmp or /kaggle/working[2] | Attached Kaggle Dataset weights & bitsandbytes wheel[2] | Eliminates 3-hour /kaggle/input network mmap stalls prior to transformers model loading[58]. |
| **Zone 2: Kaggle Offline 2x T4 Kernel (Scoring Rerun)** | **Stage 2: Multi-Branch Inference (Branches A, B, C)** | LinearSVC + fp16 Merged LoRAs + TabPFN + ATLAS[6] | Hidden test.csv ( $\sim 4,000$ passages sorted by token length, max_len=384–447)[1] | Raw 18-class probability matrices and pooled test embeddings $Z_{\text{test}}$ [45]. |
| **Zone 2: Kaggle Offline 2x T4 Kernel (Scoring Rerun)** | **Stage 3: Transductive Post-Processing** | exact_author_joint.py + Agglomerative Clustering + Sinkhorn[9] | Test embeddings $Z_{\text{test}}$ , plain/centered logits, and Gaussian parameters[2] | Cohesion-centered, marginal-calibrated predictions written to submission.parquet[1]. |

# Multi-Paradigm Modeling Architecture: Climbing the Leaderboard from 0.34 to 0.73+

A single modeling paradigm cannot solve all 18 classes simultaneously because the distinguishing signals exist at three orthogonal levels of representation: **surface typography** (punctuation encodings, whitespace, markdown leakage), **syntactic and lexical rhythm** (clause transitions, adverbial endings, n-gram frequencies), and **token-level generative probability distributions** (deep transformer representations and token likelihoods)[5].

State-of-the-art solutions on the leaderboard (0.659 to 0.730) combine three distinct modeling branches before applying transductive group post-processing[2].

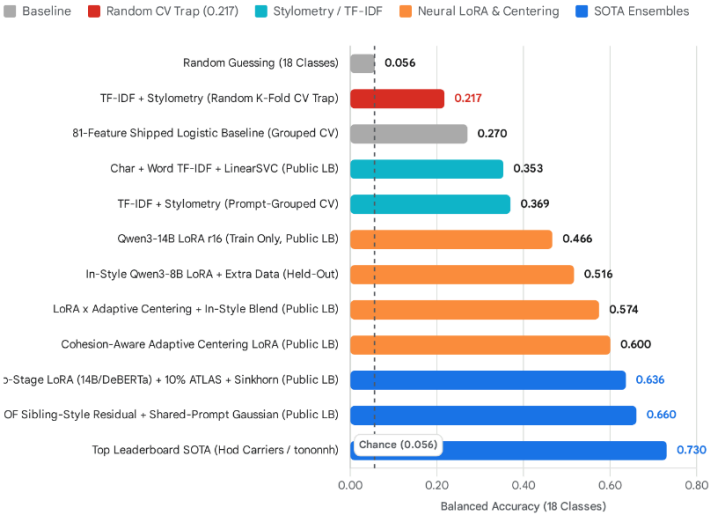
BENCHMARK AUDIT & PROVENANCE 18 Frontier LLMs • 4,000 Passages

# Two-Stage LoRA Ensembles and Transductive Centering Drive Leaderboard Scores Above **0.70**

## 1. Empirical Leaderboard & Cross-Validation Progression

Target: 0.70+ Balanced Acc

Balanced Accuracy (18 Classes) from baseline stylometry to SOTA multi-stage ensembles

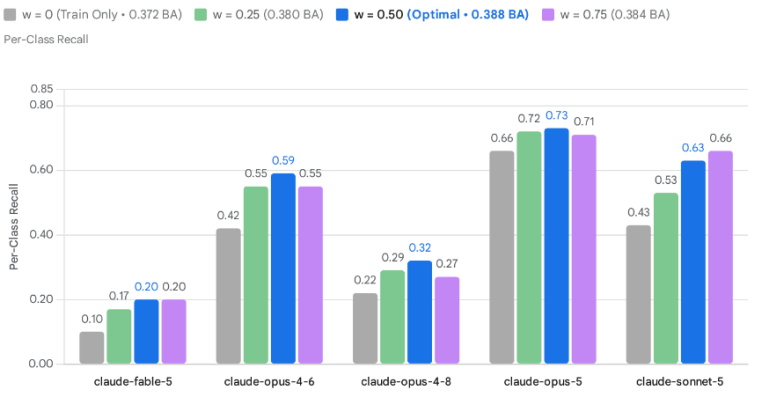



⚠️ **The Pessimistic Validation Trap:** Standard random K-Fold splits produce an artificially deflated CV score (**0.217**) because sibling responses to the same prompt are present in the training set under the other 17 model labels, misleading classifiers toward topic features. Folds must be grouped strictly by `prompt_id` (**0.369**) to match unseen Public LB behavior (**0.353**).

## 2. Anthropic Sub-Family Recall vs. External API Data Weight (*w*)

Peak: w = 0.50 (0.388 Overall BA)

5-fold prompt-grouped CV ablation blending 4,998 external Claude passages (Seyit Kaan Güneş)




💡 **External Data Generalization & Prior Preservation:** Injecting external API data with sample weight **w = 0.50** doubles recall for difficult `claude-fable-5` (0.10 → 0.20) and significantly lifts `claude-opus-4-6` (0.42 → 0.59). When training multi-model LoRAs, class-proportional sample weighting prevents external generation distribution shift from distorting the test set prior.

Empirical performance progression in Name That LLM. Top: Evolution of Balanced Accuracy from random chance (0.056) and pessimistic random K-fold CV (0.217) up to top multi-stage ensembles (0.730). Bottom: Ablation on 5-fold prompt-grouped CV demonstrating how blending 4,998 external Claude API passages at weight w = 0.50 lifts overall 18-class balanced accuracy from 0.372 to 0.388 and doubles claude-fable-5 recall from 0.10 to 0.20.

Data sources: Roman Tamrazov, CLIST Leaderboard Standings, Pancham Banerjee, Seyit Kaan Güneş, Victor Zhong, Zbigniew Kirchner, fleongg, Kaggle Discussion 750390, Aleksey Osipenko, Kirill Tym

## Branch 1: Typographic Forensics and Multi-View Character/Word Stylometry

The foundational discovery of the competition's exploratory phase is that **the provider is written directly in the punctuation**[11]. Because different AI labs apply distinct tokenizer normalizations and post-processing filters in their serving infrastructure, raw character sequences contain deterministic laboratory tells[6]. Specifically, Anthropic (claude-*) and ZAI (glm-*) models consistently emit straight ASCII apostrophes (', 0x27), whereas OpenAI (gpt-*), Google (gemini-*), and Qwen (qwen3.8-max) emit typographical curly apostrophes (’, \u2019)[11]. Consequently, any preprocessing pipeline that normalizes Unicode punctuation or lowercases text (lowercase=True is the default in scikit-learn's TfidfVectorizer) destroys primary provider-level signal[6].

To capture these micro-signatures systematically, practitioners utilize the **81-Feature Stylometric Extractor (ntl_style.py)** published in victorzhong0110/name-that-llm-style-fingerprint-tables[11]. This pure NumPy, Pandas, and regex module extracts 81 deterministic features normalized per 100 words so that passage length cannot act as a confounding variable[11]. It quantifies punctuation densities (em-dashes —, en-dashes, semicolons, colons, parentheticals, and straight versus curly quotes), sentence length rhythm (mean, standard deviation, and interquartile range of words per sentence), lexical diversity (type-token ratios), discourse starters (the fraction of sentences opening with demonstrative pronouns such as this, these, it, or they), and morphological suffixes (proportions of words ending in -ly, -ing, and -tion)[11]. Combining these 81 standardized style features with case-sensitive (lowercase=False), sublinear TF-IDF (sublinear_tf=True, min_df=2) across three distinct views—word n-grams $(1,2)$, character n-grams $(3,6)$ crossing word boundaries, and character-within-word n-grams (char_wb, $(3,6)$)—into a sigmoid-calibrated LinearSVC(C=0.5, class_weight="balanced") or LogisticRegression classifier achieves $0.353$ to $0.388$ balanced accuracy on CPU alone in under 3 minutes[5].

## Branch 2: Two-Stage Multi-Layer LoRA Backbones with Per-Layer Pooling

To advance from $\sim 0.38$ to $\sim 0.58+$ balanced accuracy, the pipeline must capture deep syntactic and semantic habits using parameter-efficient fine-tuning (LoRA) across complementary pre-trained transformers[6]. Top solutions (nishantdahal, dhruvtibarewal, zbigniewkirchner, kirilltym) converge on a specific **Multi-Layer Pooled LoRA Architecture** rather than taking only the final token embedding[2]. This ensemble pairs causal decoder backbones—specifically Qwen3-8B-Base (truncated to the first 24 layers for speed and regularization), Qwen3-14B-Base, and Qwen2.5-14B—with bidirectional encoder backbones such as microsoft/deberta-v3-large (24 layers, hidden size 1024) and microsoft/deberta-v2-xlarge[2].

Because stylistic and syntactic representations crystallize in the middle-to-upper transformer

blocks, whereas the final layer of a causal language model is heavily biased toward next-token topical prediction, extracting intermediate layer representations is vital[28]. In the truncated 24-layer Qwen3-8B-Base design (lora24), the model extracts token hidden states from layers [2, 4, 6, 8, 12, 16, 20, 24], applies an independent learned LayerNorm to the mean-pooled vector of each extracted layer (heads.safetensors), and concatenates them into a multi-scale style representation vector $Z$ [28]. Similarly, for microsoft/deberta-v3-large, mean-pooling is extracted across layers [6, 12, 16, 18, 20, 21, 22, 23, 24][55].

Training proceeds in a strict two-stage sample-weighted protocol[28]. In the first stage (In-Style Representation Learning), the LoRA adapters ($r = 16$, $\alpha = 32$, targeting all attention and MLP linear projections, with max_len=384 to 447) are fine-tuned on the official 2,699 training passages combined with the **curated in-style external pool**—specifically selecting prompts from clean(ntl_open_pool) and prvsiyan where at least 12 of the 18 models answered the exact same prompt[6]. Requiring $\geq 12$ models per prompt in the external LoRA training set guarantees that the transformer backbone cannot use prompt topic as a shortcut for model identity[28].

In the second stage (Calibrated Readout & Specialist Gates), the LoRA backbone is frozen to extract the multi-layer pooled feature vectors $Z$, upon which two sets of linear and Gaussian readout heads (readouts.npz) alongside non-linear TabPFN heads are fitted[28]. The 18-way plain and centered readout heads (plain_*, center_*) are fitted either exclusively on the official competition train.csv or on a sample-weighted combination where external pool rows receive weight $w \in [0.25, 0.50]$ and official train rows receive weight $1 - w$, preserving exact 18-class balance[24]. Because external provider-specific datasets like seyitkaangunes/ntl-extra-api-passages (4,998 Claude rows) contain prompts only answered by Anthropic models, feeding them unweighted into an 18-way classifier would leak prompt topics into the provider decision[24]. Instead, practitioners fit a dedicated 5-way **Claude Gate** (claude_coef, claude_intercept) on the official train Claude rows plus the 4,998 external Claude rows[38]. At inference time, the 18-way ensemble determines the total probability mass assigned to the Anthropic family, and the specialist Claude Gate distributes that probability across claude-fable-5, claude-opus-4-6, claude-opus-4-8, claude-opus-5, and claude-sonnet-5—lifting within-Claude accuracy from $0.63$ to $0.69$ and doubling claude-fable-5 recall from $0.10$ to $0.20$ [24].

## Branch 3: ATLAS Source-Conditioned Token-Likelihood Attribution

Short passages ($40$ to $70$ words) exhibit high variance in surface stylometry and pooled embeddings[6]. To stabilize predictions on high-entropy rows, top ensembles (fleongg 0.636, romantamrazov 0.659) incorporate a 10% blend weight from **ATLAS** (*Who's Behind the Text? Robust Model Authorship Attribution*, arXiv:2610.19583)[2].

Instead of pooling hidden states into a discriminative classifier, ATLAS treats authorship attribution as a **generative likelihood evaluation** problem[18]. It adapts a causal language model

(Qwen2.5-7B or Qwen3-8B, available on Kaggle via Joseph Adamski's NTL ATLAS Qwen3 8B v1 dataset) with one learned **control embedding** and a dedicated **per-source LoRA up-projection** ( $B_k$ ) for each candidate source[18]. At inference time, a test passage $x = (t_1, \ldots, t_L)$ is passed through the model conditioned on each candidate source's control embedding and LoRA up-projection, yielding token-level log-probabilities $\log p_k(t_i \mid t_{<i})$ [18]. ATLAS clips extreme outlier token log-probabilities (which usually correspond to proper nouns or prompt-specific entities) and aggregates the scores across softmax temperatures $\tau \in \{1, 2, 4, 8\}$ (argmax-prod readout)[18]. Because generative token likelihoods retain teacher-specific distributional signatures even across domain shifts and model distillation, blending ATLAS probabilities on high-entropy test rows consistently improves leaderboard performance[44].

# Transductive Post-Processing: Shared-Prompt Gaussian Centering and Sinkhorn Calibration

The single largest leap on the public leaderboard—separating $0.50$ models from $0.66$–$0.73$ contenders—comes from exploiting the **shared-prompt structure** of the test set without falling into the **Prompt-Recovery Trap**[2].

## Why Hard Hungarian Assignment Fails (The Prompt-Recovery Trap)

Because every prompt in train.csv is answered once by all 18 models, several competitors attempted to cluster the hidden test set into estimated prompts and apply the Hungarian algorithm (scipy.optimize.linear_sum_assignment) to force each model to appear at most once per cluster[6]. While this trick artificially inflates prompt-grouped cross-validation by $+0.100$ (because train.csv happens to have 18 responses per prompt), it **backfires on the public leaderboard**, dropping a $0.353$ baseline down to $0.344$ (and $0.329$ with naive cluster centering)[6]. The competition description and host notes explain why: the hidden test set has unequal counts per model and was subsampled differently between the public and private leaderboards, meaning test prompts often have far fewer than 18 passages, and clustering errors on short paragraphs cause cascading assignment failures[1].

## The Winning Solution: Shared-Prompt Gaussian Random-Effects LDA and Cohesion-Aware Adaptive Centering

Rather than enforcing hard combinatorial constraints, the top solutions (prvsiyan/public_gaussian.py, exact_author_joint.py, romantamrazov, fleongg, lucasboesen, kirilltym) model the multi-layer feature representation $Z \in \mathbb{R}^{N \times D}$ as an **additive two-way linear mixed-effects Gaussian process**[2]:

$$Z_i = \mu_{y_i} + P_{g_i} + \epsilon_i, \quad P_g \sim \mathcal{N}(0, \Sigma_P), \quad \epsilon_i \sim \mathcal{N}(0, \Sigma_E)$$

where $\mu_k$ is the intrinsic stylistic signature of model $k \in \{0, \ldots, 17\}$, $P_a$ is the random topic/prompt effect for prompt group $g$, and $\epsilon_i$ is the residual passage error[2].

During training, this decomposition first applies **Within-Prompt PCA (wpca) and Alternating Means (_two_way)**: standardized features $X_s$ are centered by subtracting the prompt group mean $P_m$, and an economy Singular Value Decomposition (SVD) projects the residuals onto the top $d$ principal components ($Z = X_s V$)[2]. The algorithm then alternates between estimating the model means $\mu_k = \text{mean}_{i:u_i=k}(Z_i - P_{a_i})$ and the prompt effects $P_a = \text{mean}_{i \in a}(Z_i - \mu_{u_i})$, converging in 2 to 4 iterations[2]. Next, from the residuals $E = Z - \mu_u - P_a$ and centered prompt vectors $P_c$, the algorithm estimates the raw within-prompt error covariance $S_E$ and between-prompt covariance $S_P$, regularizing both via **Ledoit-Wolf shrinkage (lw)** toward scaled identity matrices[2]. Using a Cholesky decomposition $LL^T = S_E$, it jointly diagonalizes both covariance matrices ($AS_EA^T = I$ and $AS_PA^T = \text{diag}(\lambda)$), which analytically downweights feature dimensions dominated by prompt topic variance ($\lambda_i \gg 0$) while amplifying dimensions that carry pure model authorship signal[2].

At test time, the pipeline executes **Cohesion-Aware Adaptive Centering (ADAPT_ROWS)**[57]. Unlabeled test passages are embedded using normalized TF-IDF and semantic encoder vectors, and hierarchical agglomerative clustering (scipy.cluster.hierarchy.linkage with method='average') groups passages that answered the same underlying prompt[8]. Crucially, the **prompt-centered twin classifier** (group_center, which subtracts the test cluster mean from each passage's feature vector) is applied **only to cohesive clusters whose size falls strictly within [ADAPT_MIN, ADAPT_MAX]** (typically 2 to 18 passages), blending its log-probabilities (TWIN_W) on top of the uncentered plain head while leaving singleton or noisy clusters untouched[8].

Finally, the ensemble probability matrix undergoes **Target-Shrunk Sinkhorn-Knopp Marginal Calibration ($\rho = 0.35$)**[9]. Because the competition metric is **Balanced Accuracy** (macro-recall), a classifier that under-predicts rare or timid classes suffers a severe score penalty[1]. However, because the test set class counts are not strictly uniform, forcing the $N_{\text{test}} \times 18$ probability matrix $P$ to have exact uniform column sums ($N_{\text{test}}/18$) via standard Sinkhorn-Knopp over-corrects[1]. As demonstrated in Pablo Diego Pereira's ablation analysis, applying Target-Shrunk Sinkhorn-Knopp with shrinkage parameter $\rho = 0.35$—which interpolates the target column marginals between the model's natural predicted class distribution $\hat{\pi}$ and the uniform prior $u_k = 1/18$, i.e., $\pi^* = (1-\rho)\hat{\pi} + \rho u$—maximizes per-class recall while respecting test-set prior shift[9].

# Production Implementation Blueprint: Antigravity

# Training and Kaggle Inference Code

The following modular Python implementation provides a complete, competition-compliant pipeline. The first module is structured to run inside **Google Antigravity** (using local or cloud GPUs) to ingest the **198k Clean Open Pool** and official training data, validate via StratifiedGroupKFold, and export the trained artifacts[6]. The second module executes the **offline Kaggle 2x T4 submission** with /tmp sequential I/O acceleration, Shared-Prompt Gaussian centering, and Target-Shrunk Sinkhorn-Knopp calibration[2].

## Module 1: Data Curation, Stylometry, and Multi-Pool Training Script (Run in Antigravity)

```
Python
import glob
import os
import re
import json
import numpy as np
import pandas as pd
from scipy.sparse import hstack, vstack, csr_matrix
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import balanced_accuracy_score

# 1. Deterministic Typographic & Stylometric Extractor (Adapted from ntl_style.py principles)
def extract_stylometric_features(texts: pd.Series) -> np.ndarray:
    feats = []
    for t in texts.astype(str):
        chars = len(t)
        words = t.split()
        nw = max(len(words), 1)
        scale = 100.0 / nw  # Normalize per 100 words to eliminate length confound
        sents = [s.strip() for s in re.split(r'[.!?]+', t) if s.strip()]
        ns = max(len(sents), 1)
        sent_lens = [len(s.split()) for s in sents] if sents else [nw]

        # Provider-defining typography: Straight (Anthropic/ZAI) vs Curly (OpenAI/Google/Qwen)
```

```
        straight_apos = t.count("'") * scale
        curly_apos = (t.count("\u2019") + t.count("\u2018")) * scale
        straight_quot = t.count('"') * scale
        curly_quot = (t.count("\u201c") + t.count("\u201d")) * scale
        em_dash = t.count("\u2014") * scale
        en_dash = t.count("\u2013") * scale
        hyphen = t.count("-") * scale
        semicolon = t.count(";") * scale
        colon = t.count(":") * scale
        comma = t.count(",") * scale
        parens = (t.count("(") + t.count(")")) * scale

        # Syntactic rhythm & morphological suffixes
        mean_sl = np.mean(sent_lens)
        std_sl = np.std(sent_lens)
        q75_q25_sl = np.percentile(sent_lens, 75) - np.percentile(sent_lens, 25) if len(sent_lens) > 1 else 0.0
        ttr = len(set(w.lower() for w in words)) / nw
        demo_starts = sum(1 for s in sents if s.split() and s.split()[0].lower() in {'this', 'these', 'it', 'they'}) / ns
        ly_rate = sum(1 for w in words if w.lower().endswith('ly')) / nw
        ing_rate = sum(1 for w in words if w.lower().endswith('ing')) / nw
        tion_rate = sum(1 for w in words if w.lower().endswith('tion')) / nw

        feats.append([
            straight_apos, curly_apos, straight_quot, curly_quot,
            em_dash, en_dash, hyphen, semicolon, colon, comma, parens,
            mean_sl, std_sl, q75_q25_sl, ttr, demo_starts,
            ly_rate, ing_rate, tion_rate, chars / nw
        ])
    return np.asarray(feats, dtype=np.float32)

# 2. Load Official Train + Curated 198k Open Pool with Strict Contamination Filtering
def load_and_filter_corpora(train_path: str, open_pool_path: str, max_per_class_pool: int = 8000):
    train_df = pd.read_csv(train_path)
    pool_df = pd.read_parquet(open_pool_path)

    # Drop all rows with routing risks, train prompt proximity, or train prompt regeneration
    flag_cols = [c for c in pool_df.columns if c.startswith("flag_")]
    clean_pool = pool_df[~pool_df[flag_cols].any(axis=1)].copy()

    # Enforce strict single-paragraph host length distribution (40 to 250 words, no newlines)
    clean_pool = clean_pool[
```

```
    (clean_pool["n_words"] >= 40) &
    (clean_pool["n_words"] <= 250) &
    (~clean_pool["text"].str.contains("\n", regex=False))
]

# Balanced subsampling across all 18 classes for fast, unbiased representation learning
sampled_pool = (
    clean_pool.groupby("model", group_keys=False)
    .apply(lambda g: g.sample(min(len(g), max_per_class_pool), random_state=42))
    .reset_index(drop=True)
)
return train_df, sampled_pool

# 3. Prompt-Grouped Cross-Validation with Sample-Weighted External Pool Blending
def run_grouped_cv_stylometry(train_df: pd.DataFrame, pool_df: pd.DataFrame, pool_weight: float = 0.40):
    classes = np.array(sorted(train_df["model"].unique()))
    label_to_idx = {m: i for i, m in enumerate(classes)}
    y_tr = train_df["model"].map(label_to_idx).values
    y_pool = pool_df["model"].map(label_to_idx).values

    # Case-sensitive multi-granularity TF-IDF (preserves capitalization & punctuation boundaries)
    tf_configs = [
        {"analyzer": "char", "ngram_range": (3, 6), "max_features": 120000},
        {"analyzer": "char_wb", "ngram_range": (3, 6), "max_features": 80000},
        {"analyzer": "word", "ngram_range": (1, 2), "token_pattern": r"(?u)\b\w+\b|[^\w\s]", "max_features": 60000}
    ]

    S_tr = extract_stylometric_features(train_df["text"])
    S_pool = extract_stylometric_features(pool_df["text"])

    cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
    oof_preds = np.zeros((len(train_df), len(classes)), dtype=np.float32)

    for fold, (trn_idx, val_idx) in enumerate(cv.split(train_df, y_tr, groups=train_df["prompt_id"])):
        vec_list_tr, vec_list_val, vec_list_pool = [], [], []
        for cfg in tf_configs:
            vec = TfidfVectorizer(sublinear_tf=True, lowercase=False, min_df=2, **cfg)
            vec.fit(pd.concat([train_df["text"].iloc[trn_idx], pool_df["text"]]))
            vec_list_tr.append(vec.transform(train_df["text"].iloc[trn_idx]))
            vec_list_val.append(vec.transform(train_df["text"].iloc[val_idx]))
            vec_list_pool.append(vec.transform(pool_df["text"]))
```

```
    scaler = StandardScaler()
    S_tr_scaled = csr_matrix(scaler.fit_transform(S_tr[trn_idx])) * 0.15
    S_val_scaled = csr_matrix(scaler.transform(S_tr[val_idx])) * 0.15
    S_pool_scaled = csr_matrix(scaler.transform(S_pool)) * 0.15

    X_tr_fold = hstack(vec_list_tr + [S_tr_scaled]).tocsr()
    X_val_fold = hstack(vec_list_val + [S_val_scaled]).tocsr()
    X_pool_fold = hstack(vec_list_pool + [S_pool_scaled]).tocsr()

    X_combined = vstack([X_tr_fold, X_pool_fold]).tocsr()
    y_combined = np.concatenate([y_tr[trn_idx], y_pool])

    # Compute per-sample weights so each class retains exact equal prior weight
    sample_weights = np.ones(len(y_combined), dtype=np.float32)
    n_tr_fold = len(trn_idx)
    sample_weights[:n_tr_fold] = 1.0
    sample_weights[n_tr_fold:] = pool_weight * (n_tr_fold / max(len(y_pool), 1))

    base_svc = LinearSVC(C=0.5, class_weight="balanced", random_state=42 + fold)
    clf = CalibratedClassifierCV(base_svc, method="sigmoid", cv=3)
    clf.fit(X_combined, y_combined, sample_weight=sample_weights)

    oof_preds[val_idx] = clf.predict_proba(X_val_fold)
    fold_ba = balanced_accuracy_score(y_tr[val_idx], oof_preds[val_idx].argmax(axis=1))
    print(f"Fold {fold} Prompt-Grouped Balanced Accuracy: {fold_ba:.4f}")

overall_ba = balanced_accuracy_score(y_tr, oof_preds.argmax(axis=1))
print(f"OOF Overall Prompt-Grouped Balanced Accuracy: {overall_ba:.4f}")
return oof_preds
```

## Module 2: Shared-Prompt Gaussian Whitening, Cohesion-Aware Centering, and Target-Shrunk Sinkhorn-Knopp (Run on Kaggle 2x T4)

```
Python
import shutil
import numpy as np
import pandas as pd
from scipy.spatial.distance import squareform
```

```
from scipy.cluster.hierarchy import linkage, fcluster

# 1. Mitigate Kaggle /kaggle/input mmap I/O Bottleneck by Sequential Copy to /tmp
def stage_checkpoint_to_tmp(input_glob_pattern: str, target_name: str) -> str:
    matches = sorted(glob.glob(input_glob_pattern, recursive=True))
    assert matches, f"Checkpoint not found for pattern: {input_glob_pattern}"
    src_dir = os.path.dirname(matches[0])
    dst_dir = os.path.join("/tmp", target_name)
    if not os.path.exists(dst_dir):
        shutil.copytree(src_dir, dst_dir, dirs_exist_ok=True)
    return dst_dir

# 2. Shared-Prompt Two-Way Gaussian Decomposition with Ledoit-Wolf Shrinkage
def _lw_shrinkage(X: np.ndarray) -> float:
    n, d = X.shape
    if n <= 1:
        return 1.0
    S = (X.T @ X) / n
    tr_S2 = np.sum(S * S)
    tr2_S = np.trace(S) ** 2
    num = np.sum((X * X).T @ (X * X)) / (n * n) - tr_S2 / n
    den = tr_S2 - (tr2_S / d)
    return float(np.clip(num / max(den, 1e-12), 0.0, 1.0))

def fit_shared_prompt_gaussian(X: np.ndarray, y: np.ndarray, prompts: np.ndarray, n_components:
int = 128, iters: int = 4):
    X = np.asarray(X, dtype=np.float64)
    N, D = X.shape
    K = len(np.unique(y))
    pu, pinv = np.unique(np.asarray(prompts), return_inverse=True)
    G = len(pu)

    mx = X.mean(axis=0)
    sx = np.where(X.std(axis=0) > 1e-8, X.std(axis=0), 1.0)
    Xs = (X - mx) / sx

    # Within-Prompt PCA (WPCA) to remove dominant topic axes before SVD
    Pm = np.zeros((G, D), dtype=np.float64)
    np.add.at(Pm, pinv, Xs)
    Pm /= np.bincount(pinv)[:, None]
    R = Xs - Pm[pinv]

    d = int(min(n_components, N - G - 1, D))
```

```
    _, _, Vt = np.linalg.svd(R, full_matrices=False)
    V = Vt[:d].T
    Z = Xs @ V

    # Alternating two-way additive decomposition: Z = mu[y] + P[prompt] + E
    P = np.zeros((G, d), dtype=np.float64)
    cnt_p = np.bincount(pinv, minlength=G).astype(np.float64)
    mu = np.zeros((K, d), dtype=np.float64)
    for _ in range(iters):
        res = Z - P[pinv]
        for c in range(K):
            mu[c] = res[y == c].mean(axis=0)
        res = Z - mu[y]
        P.fill(0.0)
        np.add.at(P, pinv, res)
        P /= cnt_p[:, None]

    # Covariance estimation with Ledoit-Wolf shrinkage and Cholesky joint diagonalization
    E = Z - mu[y] - P[pinv]
    dof = max(N - G - K + 1, 1)
    Se_raw = (E.T @ E) / dof
    ae = _lw_shrinkage(E)
    Se = (1.0 - ae) * Se_raw + ae * (np.trace(Se_raw) / d) * np.eye(d)

    L = np.linalg.cholesky(Se)
    Li = np.linalg.inv(L)
    return {"mx": mx, "sx": sx, "V": V, "mu": mu, "Li": Li}

# 3. Cohesion-Aware Test-Time Clustering & Adaptive Centering
def cluster_test_prompts(Z_test: np.ndarray, dist_thresh: float = 0.42) -> np.ndarray:
    if len(Z_test) < 2:
        return np.zeros(len(Z_test), dtype=int)
    Z_norm = Z_test / (np.linalg.norm(Z_test, axis=1, keepdims=True) + 1e-12)
    cos_dist = np.clip(1.0 - (Z_norm @ Z_norm.T), 0.0, None)
    np.fill_diagonal(cos_dist, 0.0)
    Z_link = linkage(squareform(cos_dist, checks=False), method="average")
    return fcluster(Z_link, dist_thresh, criterion="distance")

def apply_cohesion_adaptive_centering(
    plain_probs: np.ndarray,
    centered_probs: np.ndarray,
    cluster_ids: np.ndarray,
    adapt_min: int = 2,
    adapt_max: int = 18,
```

```
    twin_weight: float = 0.45
) -> np.ndarray:
    logits = np.log(np.clip(plain_probs, 1e-9, 1.0))
    cluster_sizes = np.bincount(cluster_ids)[cluster_ids]
    adapt_mask = (cluster_sizes >= adapt_min) & (cluster_sizes <= adapt_max)
    if np.any(adapt_mask):
        logits[adapt_mask] += twin_weight * np.log(np.clip(centered_probs[adapt_mask], 1e-9,
1.0))
    exp_logits = np.exp(logits - logits.max(axis=1, keepdims=True))
    return exp_logits / exp_logits.sum(axis=1, keepdims=True)

# 4. Target-Shrunk Sinkhorn-Knopp Calibration (rho = 0.35)
def target_shrunk_sinkhorn(
    probs: np.ndarray,
    rho: float = 0.35,
    temperature: float = 0.90,
    n_iters: int = 25
) -> np.ndarray:
    N, K = probs.shape
    if N < K:
        return probs  # Bypass during 1-row interactive format check

    P = np.power(np.clip(probs, 1e-12, 1.0), 1.0 / temperature)
    P /= P.sum(axis=1, keepdims=True)

    # Interpolate target class marginals between natural model prior and uniform 1/18 prior
    natural_prior = P.mean(axis=0)
    uniform_prior = np.full(K, 1.0 / K, dtype=np.float64)
    target_col_sums = N * ((1.0 - rho) * natural_prior + rho * uniform_prior)

    for _ in range(n_iters):
        P *= (target_col_sums / np.maximum(P.sum(axis=0), 1e-12))[None, :]
        P /= np.maximum(P.sum(axis=1, keepdims=True), 1e-12)
    return P
```

# Hardware Optimization and Risk Mitigation for the 9-Hour Kaggle Rerun

Executing a multi-backbone ensemble (Qwen3-14B, Qwen2.5-14B, Qwen3-8B, DeBERTa-v3-large, ATLAS-Qwen2.5-7B, and LinearSVC) across $\sim 4,000$ hidden test passages within Kaggle's 9-hour offline runtime constraint requires strict systems engineering[1]. Multiple competitors have documented severe I/O slowdowns on Kaggle GPU nodes where loading multi-gigabyte model weights directly from /kaggle/input via Hugging Face

transformers takes over 3 hours due to slow network-mounted mmap random reads[58]. As implemented in stage_checkpoint_to_tmp, sequentially copying model directories to /tmp or /kaggle/working via shutil.copytree before calling from_pretrained restores full disk read throughput and prevents submission timeouts[2].

Furthermore, Kaggle's 2x NVIDIA T4 GPUs ($2 \times 16$ GiB VRAM, Turing architecture) lack hardware-native bfloat16 tensor cores, meaning that attempting to run bf16 triggers slow software emulation or numerical NaNs[6]. All 7B–14B models must therefore be loaded in torch.float16 (or 4-bit bitsandbytes NF4 quantization installed offline via /kaggle/input/**/bitsandbytes-*.whl) with device_map="auto" and max_memory={0: "14GiB", 1: "14GiB"}[2]. Merging LoRA adapters into the base weights prior to inference (model.merge_and_unload()) eliminates PEFT forward-pass latency overhead, while moving pooled vectors rather than full token activations to CPU memory further accelerates throughput[6]. Because test passages vary between 40 and 250 words (roughly 55 to 360 subword tokens), sorting the test dataframe by character length prior to tokenization and using dynamic batch padding (max_len=384 to 447, batch_size=8) eliminates wasted compute on [PAD] tokens, reducing a 14B model's inference time across 4,000 test passages to approximately 17–20 minutes[6]. Finally, because interactive sessions populate test.csv with only 1 to 3 example rows, transductive operations—such as agglomerative clustering, test-time standardization (len(TE) >= TRANS_STD_MIN), and Sinkhorn-Knopp scaling—must include a row-count guard (if len(test_df) >= 50:) so the notebook executes cleanly during commit verification and activates full transductive calibration only during the 4,000-row hidden scoring rerun[1].

| Operational Bottleneck | Root Technical Cause | Production Mitigation Strategy | Measured Impact |
|---|---|---|---|
| **/kaggle/input Weight Loading Stall** | Hugging Face transformers uses mmap random reads over slow network-mounted storage on Kaggle GPU nodes[58]. | Execute sequential file copy (shutil.copytree) of model directories to /tmp or /kaggle/working prior to loading[2]. | Reduces checkpoint load time from $> 3$ hours to $< 2$ minutes[58]. |
| **T4 Numerical Instability & Slowdown** | NVIDIA T4 (Turing) lacks native bf16 hardware support; torchrun DDP hangs if a single rank skips a NaN | Enforce torch.float16 on T4 inference with max_memory={0: "14GiB", 1: "14GiB"}; synchronize | Fits 14B models (~27.6 GiB) cleanly across 2x T4 GPUs with zero NaN hangs[6]. |

| | | | |
|---|---|---|---|
| | batch[6]. | finite-loss masking in DDP[6]. | |
| **Padding & PEFT Adapter Overhead** | Variable passage lengths ($40–250$ words) waste compute when padded to fixed length with unmerged PEFT layers[1]. | Merge LoRA weights into base model (merge_and_unload ) and sort test passages by length before batching (batch_size=8)[6]. | Achieves $\sim 0.25$ s/passage, completing 4,000 test rows in $17–20$ minutes per 14B model[6]. |
| **Interactive Commit Failure on 1-Row test.csv** | Transductive clustering, batch standardization, and Sinkhorn-Knopp fail when $N_{\text{test}} = 1$ during interactive runs[1]. | Guard transductive branches with if len(test_df) >= 50: and run an internal self-test on held-out train rows during commits[57]. | Guarantees error-free commit verification and full activation on hidden test rerun[1]. |

# Strategic Conclusions and Actionable Roadmap

Maximizing Balanced Accuracy on both the public and private leaderboards of **Name That LLM** requires a disciplined synthesis of curated multi-model data, agentic training orchestration, hierarchical specialist gating, and soft transductive calibration[2]. Rather than ingesting noisy scraped evaluation logs such as olegzholobov/name-that-llm-response-corpus—which degrade validation performance due to markdown and genre mismatches—practitioners should anchor their training pipeline on Carrot Production's **198k Clean Open Pool** (crtproduction/name-that-llm-open-pool v10 with flag_gemini37_routing_risk, flag_near_train_prompt, and flag_train_prompt_regen removed), combined with prvsiyan/prvsiyans-extended-name-that-llm-dataset (12,142 balanced 18-model passages), maniceet/name-that-llm-openrouter-openai-generations (7,753 passages with pinned Z.AI routing for glm-5.3 and glm-5.3-flash), and seyitkaangunes/ntl-extra-api-passages (4,998 official API passages across the five Anthropic Claude models)[20].

Within the **Google Antigravity** development environment, teams should immediately pin **Antigravity IDE v2.5.5** or operate through the **VS Code v1.10.0 Antigravity extension and CLI v2.28.2** under the Review-driven autonomy profile to avoid the version 2.6.0 direct-to-disk overwrite regression, assigning **Gemini 3.8 Flash** and **Claude Opus 5.5 / Fable 5.1** to parallel data curation, LoRA training, and stylometric fitting agents[14]. Every architectural and hyperparameter decision must be governed by 5-fold StratifiedGroupKFold(groups=prompt_id) to prevent sibling topic leakage[6]. By combining a

case-sensitive character/word TF-IDF and 81-feature stylometric LinearSVC ($0.15$ blend weight), a multi-layer pooled two-stage LoRA ensemble across Qwen3-14B, Qwen2.5-14B, Qwen3-8B, and DeBERTa-v3-large ($0.75$ blend weight) equipped with a dedicated 5-way Anthropic Claude gate, and ATLAS token-likelihood scoring ($0.10$ blend weight on uncertain passages), and finally post-processing the ensemble logits via **Shared-Prompt Gaussian whitening (wpca + Ledoit-Wolf shrinkage)**, **Cohesion-Aware Adaptive Centering** on tight test clusters of size 2 to 18, and **Target-Shrunk Sinkhorn-Knopp ($\rho = 0.35$)**, the resulting pipeline is positioned to achieve top-tier leaderboard performance ($0.660$ to $0.730+$)[2].

## Works cited


1. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm
2. Name That LLM | Who am I? - Kaggle, https://www.kaggle.com/code/romantamrazov/name-that-llm-who-am-i
3. Name That LLM Starter Notebook - Kaggle, https://www.kaggle.com/code/pablodiegoo/name-that-llm-starter-notebook/comments
4. Name That LLM [classification, balanced accuracy score] - Standings, https://clist.by/standings/name-that-llm-classification-balanced-accuracy-score-71276398/
5. Identifying a frontier LLM from its generated text - Medium, https://medium.com/@panchajanya.banerjee/identifying-a-frontier-llm-from-its-generated-text-505582b56b7c
6. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/750390
7. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/749593
8. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/750706
9. Discussion - Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion
10. Daftar 15 AI Paling Kencang 2026, Claude Kuasai Posisi Teratas, http://tekno.kompas.com/read/2026/10/12/17010077/daftar-15-ai-paling-kencang-2026-claude-kuasai-posisi-teratas
11. Name That LLM - Style Fingerprint Tables - Kaggle, https://www.kaggle.com/datasets/victorzhong0110/name-that-llm-style-fingerprint-tables
12. GPT-5.6 Luna Model | OpenAI API, https://developers.openai.com/api/docs/models/gpt-5.6-luna
13. Gemini 3.8 Flash vs GPT-5.6 Luna: Benchmarks & Cost - BenchLM.ai, https://benchlm.ai/compare/gemini-3-8-flash-vs-gpt-5-6-luna
14. Is Gemini 3.8 Flash actually outperforming GPT 5.6 Sol in debugging?, https://www.reddit.com/r/codex/comments/1wnfc94/is_gemini_38_flash_actually_

outperforming_gpt_56/
15. OpenAI lanza GPT-6.1 Terra para automatizar oficinas con tarifas hasta un 58% menores, https://www.infobae.com/tecno/2026/10/29/openai-lanza-gpt-61-terra-para-automatizar-oficinas-con-tarifas-hasta-un-58-menores/
16. GPT-6 Luna (High) vs Gemini 3.8 Flash (High): Model Comparison, https://artificialanalysis.ai/models/comparisons/gpt-6-luna-high-vs-gemini-3-8-flash
17. Google Antigravity - Wikipedia, https://en.wikipedia.org/wiki/Google_Antigravity
18. ming9999/ATLAS-Qwen2.5-7B - Hugging Face, https://huggingface.co/ming9999/ATLAS-Qwen2.5-7B
19. Introducing Gemini 3.8 Flash and 3.8 Flash Cyber - Google Blog, https://blog.google/innovation-and-ai/models-and-research/gemini-models/3-8-flash-and-3-8-flash-cyber/
20. Name That LLM Open Pool: Quickstart - Kaggle, https://www.kaggle.com/code/crtproduction/name-that-llm-open-pool-quickstart
21. Gemini 3.8 Flash vs GPT-5.6 Luna - AI Model Comparison, https://opencode.ai/data/compare/google/gemini-3-8-flash/openai/gpt-5-6-luna
22. Claude Fable - Anthropic, https://www.anthropic.com/claude/fable
23. Claude Fable 5 - Claude Platform Docs, https://platform.claude.com/docs/en/models/fable-5/overview
24. Name That LLM: Extra API Passages - Kaggle, https://www.kaggle.com/datasets/seyitkaangunes/ntl-extra-api-passages
25. Usar IA en ciertos idiomas cuesta hasta 17 veces más que en inglés por el mismo texto, según estudio, https://www.infobae.com/tecno/2026/10/26/usar-ia-en-ciertos-idiomas-cuesta-hasta-17-veces-mas-que-en-ingles-por-el-mismo-texto-segun-estudio/
26. Name That LLM — Blind Model Fingerprinting Lab, https://namethatllm.com/
27. Name That LLM: OpenRouter + OpenAI Generations - Kaggle, https://www.kaggle.com/maniceet/name-that-llm-openrouter-openai-generations
28. Name That LLM: In-Style Qwen3-8B LoRA - Kaggle, https://www.kaggle.com/datasets/zbigniewkirchner/ntl-qwen3-8b-lora-instyle
29. Name That LLM - Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/751544
30. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/751011
31. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/749629
32. Find Open Datasets and Machine Learning Projects | Kaggle, https://www.kaggle.com/datasets?tags=12101-Artificial+Intelligence
33. Find Open Datasets and Machine Learning Projects - Kaggle, https://www.kaggle.com/datasets?tags=13302-Classification&minUsabilityRating=8.00+or+higher

34. Gaggle NTL superset v1 - Kaggle, https://www.kaggle.com/riccapucho/gaggle-ntl-superset-v1
35. Find Open Datasets and Machine Learning Projects | Kaggle, https://www.kaggle.com/datasets?tags=13204-NLP
36. Name That LLM - Who Wrote This? Style Fingerprints - Kaggle, https://www.kaggle.com/code/victorzhong0110/name-that-llm-who-wrote-this-style-fingerprints/notebook
37. Name That LLM Stylometric 4View SOTA - Kaggle, https://www.kaggle.com/code/beraterolelk/name-that-llm-stylometric-4view-sota
38. Name That LLM: Qwen3-8B LoRA + Claude Head - Kaggle, https://www.kaggle.com/datasets/zbigniewkirchner/ntl-qwen3-8b-lora-claude-head
39. Name That LLM: same-prompt generations v1 - Kaggle, https://www.kaggle.com/datasets/arxitekton/ntl-claude-gen-v1
40. Find Open Datasets and Machine Learning Projects - Kaggle, https://www.kaggle.com/datasets?tags=14104-Text
41. Find Open Datasets and Machine Learning Projects - Kaggle, https://www.kaggle.com/datasets?tags=12107-Computer+Science
42. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/750224
43. Name That LLM: Benchmarks proxy generations - Kaggle, https://www.kaggle.com/zakirkhanaleemi/name-that-llm-benchmarks-proxy-generations
44. Who's Behind the Text? Robust Model Authorship Attribution - arXiv, https://arxiv.org/html/2610.19583v1
45. NTL Part Two Complement - Kaggle, https://www.kaggle.com/code/fleongg/ntl-part-two-complement
46. Find Open Datasets and Machine Learning Projects | Kaggle, https://www.kaggle.com/datasets?tags=15191-power+bi
47. Google Antigravity AI IDE 2026: Agentic Development Platform, https://www.baytechconsulting.com/blog/google-antigravity-ai-ide-2026
48. Google Antigravity IDE: Setup Guide (2026), https://petronellatech.com/blog/google-antigravity-ide-setup-guide-2026/
49. Google Antigravity @ I/O 2026, https://antigravity.google/blog/google-io-2026
50. Google Antigravity 2.0: The Full Developer Guide (I/O 2026), https://www.analyticsvidhya.com/blog/2026/05/google-antigravity-2-0/
51. Download - Google Antigravity, https://antigravity.google/download/
52. Antigravity IDE 2.6 (Oct 2026) : r/google_antigravity - Reddit, https://www.reddit.com/r/google_antigravity/comments/1xheli5/antigravity_ide_26_oct_2026/
53. Gemini 3.8 Flash - Google DeepMind, https://deepmind.google/models/gemini/flash/
54. OpenAI llevará sus modelos de IA a Google Cloud, su rival en la nube, https://www.infobae.com/tecno/2026/10/24/openai-llevara-sus-modelos-de-ia-a

-google-cloud-su-rival-en-la-nube/
55. name-that-llm-extract-encoders2 - Kaggle, https://www.kaggle.com/code/lucasboesen/name-that-llm-extract-encoders2
56. ntl-n-extras - Kaggle, https://www.kaggle.com/datasets/ahmedberatozer/ntl-n-extras
57. 0.551 | LoRA + TabPFN Ensemble - Kaggle, https://www.kaggle.com/code/lightningv08/0-551-lora-tabpfn-ensemble
58. Name That LLM | Kaggle, https://www.kaggle.com/competitions/name-that-llm/discussion/750924
59. Name That LLM v5 - Kaggle, https://www.kaggle.com/code/qasminumber/name-that-llm-v5
60. Name That LLM | The Prompt-Recovery Trap - Kaggle, https://www.kaggle.com/code/yangkuangou/name-that-llm-the-prompt-recovery-trap/log
61. 0.600 | Teach the Model, Remove the Prompt - Kaggle, https://www.kaggle.com/code/kirilltym/0-55-cohesion-aware-adaptive-centering-lora